Design and Analysis of Algorithms

ASSIGNMENT 1 (CO1)

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Field	Details
Name	Poojitha
Register No.	192511230
Programme	B.E. Computer Science and Engineering
Institution	SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai
Total marks	50 (Tasks 1–11 = 42, Task 12 = 8)

Summary of the work. Three distinct strategies are designed and analysed for locating a registration number: **(A) linear search** (iterative, brute force, no preprocessing), **(B) binary search** (recursive, decrease-and-conquer, needs sorting), and **(C) hashing with chaining** (space–time trade-off, needs a hash table build). All three were implemented in Python and tested on 1,000 → 10,000 → 100,000 → 1,000,000 records. The theory (Tasks 1–7) is validated against measured data (Tasks 8–10), and a final design is recommended (Task 11) and then re-evaluated for the changed workload of 50,000 searches per hour (Task 12).

Note on the experimental numbers. All timings in this document were measured with the program in Appendix A (Python 3, single thread). Absolute times depend on the machine, so re-run the program on your own computer and replace the tables if your numbers differ; the growth trends and comparison counts will be the same.

Task 1 – Design the Algorithm (5 marks)

Problem. Given a collection of n student records, each with a unique registration number, decide whether a given key k exists and return its record. Let n range from 10^3 to 10^6 and beyond. Three different design strategies are chosen so that each makes a different trade-off between preprocessing effort and per-search effort.

	A. Linear search	**B. Binary search**	**C. Hash table (chaining)**
Design strategy	Brute force	Decrease-and-conquer (decrease by half)	Space–time trade-off / input enhancement (hashing)
Style used	Iterative	Recursive (iterative form also coded only to measure recursion overhead)	Iterative
Preprocessing	None	Sort records by registration number	Build table of m buckets; insert every record
Extra memory	O(1)	O(1) (sort in place)	O(n + m) for buckets and chain nodes

A. Linear (sequential) search – iterative

Basic idea. Examine the records one by one from the start until the key is found or the end is reached. No assumption is made about the order of the data.

Preprocessing. None; the data is used as it is stored.

```
ALGORITHM LinearSearch(A[0..n-1], key)
    for i <- 0 to n-1 do
        if A[i] = key then          // basic operation
            return i
    return -1                       // not found
```

Justification. It is the baseline: the simplest correct solution, needs no preprocessing and no extra memory, and is the best choice for very small n. It also shows exactly how bad the cost becomes when n reaches millions, which motivates the other two approaches.

B. Binary search – recursive

Basic idea. If the records are sorted by registration number, compare the key with the middle record. If equal, stop; if the key is smaller, search only the left half; otherwise search only the right half. Each comparison discards half of the remaining records.

Preprocessing. Sort the array by registration number once (merge sort / Timsort, $\Theta(n \log n)$). Sorting is required for correctness and is included in all cost analysis.

```
ALGORITHM BinarySearch(A[0..n-1], key, lo, hi)   // call with lo = 0, hi = n-1
   if lo > hi then return -1                  // base case: empty range
   mid <- floor((lo + hi) / 2)
   if A[mid] = key then return mid            // basic operation
   else if key < A[mid] then
      return BinarySearch(A, key, lo, mid-1)
   else
      return BinarySearch(A, key, mid+1, hi)
```

Justification. Registration numbers are comparable keys, so sorted order can be exploited. The cost per search drops from linear to logarithmic at the price of a one-time sort. The recursive form is chosen because the halving structure gives a clean recurrence relation (Tasks 6 and 7).

C. Hashing with separate chaining – iterative

Basic idea. Use the key itself to compute where the record should be stored. A hash function h(k) = k mod m maps each registration number to one of m buckets; records that fall in the same bucket are kept in a short list (chain). A search computes h(k) and scans only that one chain.

Preprocessing. Allocate m = n buckets (load factor $\alpha = n/m = 1$) and insert all n records.

```
ALGORITHM BuildTable(A[0..n-1], m)
   T[0..m-1] <- empty lists
   for i <- 0 to n-1 do
      append A[i] to list T[A[i] mod m]
   return T

ALGORITHM HashSearch(T, m, key)
   for each record r in list T[key mod m] do
      if r = key then return r              // basic operation
   return -1
```

Justification. It does not depend on the order of the data and aims at constant-time lookup, so it is the natural candidate for the largest databases and for a read-heavy workload. It is fundamentally different from the other two: it avoids comparing the key against many records at all.

Note. Writing binary search iteratively (or linear search recursively) would not count as a new approach; the iterative binary search in Appendix A is used only to measure the overhead of recursion.

Task 2 – Identify the Basic Operation (2 marks)

The basic operation is the operation executed most often and inside the innermost loop/recursion, so that the running time is proportional to its count. For all three algorithms the dominant operation is the **key comparison** between the search key and a stored registration number, but it occurs in a different place and grows differently in each.

Algorithm	**Basic operation and where it occurs**	**Why it is suitable**	**How its count changes as n grows**
A. Linear	Comparison A[i] = key, inside the for-loop body.	Every other statement (loop counter increment, return) executes at most once per comparison, so total time ∝ number of comparisons.	Grows in direct proportion to n: up to n comparisons; 10 times more records means about 10 times more comparisons.
B. Binary	Comparison A[mid] = key (with the < test), once per call/iteration.	Each recursive call performs exactly one probe plus O(1) arithmetic; the number of calls equals the number of comparisons.	Grows with $\log_2 n$: each doubling of n adds only one comparison.
C. Hashing	Comparison r = key while scanning the chain T[h(key)]. (Computing h(key) is one constant-cost step per search.)	The hash computation is O(1) and identical for every search, so the variable cost is the chain scan.	Depends on the load factor $\alpha = n/m$, not on n itself; with m proportional to n it stays about constant (≈ 1 to 1.5 comparisons).

For preprocessing, the basic operation is a comparison for sorting (B) and an insertion (one array-append) for the hash table (C).

Task 3 – Derive the Time-Complexity Function (6 marks)

Input size n = number of student records. C(n) denotes the number of basic operations (key comparisons) for one search; P(n) is the preprocessing cost; for q searches the total is $T(n, q) = P(n) + q \cdot C(n)$.

A. Linear search

The loop body executes once for each index examined. A key stored at position i (1-based) needs exactly i comparisons; an absent key needs n.

$$\text{Worst case: } C(n) = 1 + 1 + \ldots + 1 \text{ (n times)} = n \qquad \text{Best case: } C(n) = 1$$

Assume the key is present and equally likely to be at any of the n positions (probability 1/n each):

$$\text{Average: } C(n) = (1 + 2 + \ldots + n) \cdot (1/n) = (1/n) \cdot n(n+1)/2 = (n + 1)/2$$

If a fraction p of searches succeed and (1 – p) fail: C(n) average = $p(n+1)/2 + (1 – p)n$. Time $T(n) \approx c \cdot C(n)$, so the running time is a linear function of n. **Preprocessing:** P(n) = 0. **Overall for q searches:** $T(n, q) = q \cdot (n+1)/2$ on average (and q·n worst case) = $\Theta(q \cdot n)$.

B. Binary search

After i comparisons the unsearched range has at most $n/2^i$ records. The search must stop when the range is empty, i.e. when $n/2^i < 1$, which gives $i > \log_2 n$. The largest number of probes is therefore:

$$\text{Worst case: } C(n) = \lfloor \log_2 n \rfloor + 1$$

For n = 1,000,000: ⌊19.93⌋ + 1 = 20 comparisons. **Average (successful).** For $n = 2^k – 1$ the probe sequence forms a complete binary tree: one record needs 1 comparison, two need 2, four need 3, …, $2^{k–1}$ need k. Hence

$$\text{Average: } C(n) = (1/n) \cdot [1 \cdot 2^0 + 2 \cdot 2^1 + \ldots + k \cdot 2^{k-1}] = ((k – 1) \cdot 2^k + 1)/n \approx k – 1 \approx \log_2 n – 1$$

For $n = 10^6$ this predicts about 18.9 comparisons, which matches the measured 18.94 in Task 9. **Preprocessing.** Merge sort satisfies $S(n) = 2S(n/2) + (n – 1)$, whose worst case is $n \cdot \log_2 n – n + 1$ comparisons, so $P(n) = \Theta(n \log n)$; for $n = 10^6$ that is about 1.9×10^7 comparisons. **Overall for q searches:**

$$T(n, q) = n \cdot \log_2 n + q \cdot \log_2 n = (n + q) \cdot \log_2 n$$

C. Hash table with chaining

Preprocessing. Each of the n records is hashed (one mod operation) and appended to a chain (O(1)): $P(n) = n \cdot c_1 = \Theta(n)$. **Search.** Let m be the number of buckets and $\alpha = n/m$ the load factor. Assuming simple uniform hashing (each key equally likely to go to any bucket independently), the expected chain length is $n/m = \alpha$, so:

$$\text{Unsuccessful: } C(n) = \alpha \qquad \text{Successful: } C(n) = 1 + \alpha/2 - \alpha/(2n) \approx 1 + \alpha/2$$

With $m = n$ ($\alpha = 1$): about 1 comparison for an absent key and about 1.5 for a present key, independent of n. Including the hash computation, $C(n) = \Theta(1 + \alpha) = \Theta(1)$. In the worst case all n keys collide into one bucket and the search degrades to a linear scan, C(n) worst = n. **Overall for q searches:** $T(n, q) = \Theta(n) + q \cdot \Theta(1) = \Theta(n + q)$ on average.

Summary of the derived cost functions

Algorithm	Preprocessing P(n)	Search C(n) average	Search C(n) worst	Total for q searches
A. Linear	0	$(n + 1)/2$	n	$q \cdot n/2 \rightarrow \Theta(q \cdot n)$
B. Binary	$n \log_2 n$	$\approx \log_2 n - 1$	$\lfloor \log_2 n \rfloor + 1$	$n \log_2 n + q \log_2 n$
C. Hash ($\alpha = 1$)	n	≈ 1.5 (found) / 1 (absent)	n (all keys collide)	n + q (expected)

Break-even point (theory). Binary search pays back its sort once $q \cdot (n+1)/2 > n \cdot \log_2 n + q \cdot \log_2 n$, i.e. after about $q \approx 2 \cdot \log_2 n$ searches. For $n = 10^6$ this is only about 40 searches, so preprocessing is justified for any realistic workload. Hashing breaks even against linear search after about $q \approx 2$ searches in terms of comparisons (n extra insertions vs about n/2 saved per search).

Task 4 – Best, Average and Worst-Case Behaviour (4 marks)

A. Linear search

- **Best case (1 comparison):** the key is the first record. It does not depend on n, because the loop exits at the first iteration.
- **Average case ((n+1)/2):** the key is present at a uniformly random position, so half of the list is scanned on average. If failed searches are common, the average rises towards n.

- **Worst case (n):** the key is the last record or is not present at all; the loop must examine every record before it can conclude. A failed search is the worst case because linear search cannot know the key is absent without looking at everything.

B. Binary search

- **Best case (1):** the key sits exactly at the middle position of the initial range, so the very first probe succeeds.
- **Average case ($\approx \log_2 n - 1$):** most records are deep in the implicit search tree: half of all records are at the bottom level, a quarter one level above, and so on. A random present key is therefore found only about one comparison before the worst case.
- **Worst case ($\lfloor \log_2 n \rfloor + 1$):** the key is absent, or is found only after the range has shrunk to a single element. The range is halved every step, so the number of steps cannot exceed the number of times n can be halved. Unlike linear search, the worst case grows extremely slowly.

C. Hashing with chaining

- **Best case (1, or 0 comparisons):** the bucket holds only the target key (first in its chain) or the bucket is empty for an absent key.
- **Average case ($1 + \alpha/2$ found, α not found):** with a good hash function, keys spread evenly; each chain has $\alpha = n/m$ records on average. Because m grows with n, α stays constant.
- **Worst case (n):** every key hashes to the same bucket (e.g. all registration numbers are multiples of m when $h(k) = k \bmod m$), so the chain is the entire data set and hashing degenerates into linear search. The risk comes from the data and the choice of m, not from the size of n; using a prime m (or a better hash function) avoids it in practice.

Experimental evidence for these cases (n = 1,000,000): linear best = 1 comparison, linear worst = 1,000,000 comparisons (last key); binary worst observed = 20 comparisons ($\lfloor \log_2 10^6 \rfloor + 1 = 20$); the longest hash chain observed was 8 records, against an average of about 1.

Task 5 – Asymptotic Analysis (3 marks)

Algorithm	Best	Average	Worst	Preprocessing	Remarks
A. Linear	$\Omega(1)$	$\Theta(n)$	$\Theta(n)$	none	Overall O(n); since the best case is $\Theta(1)$ no single Θ covers all inputs.

Algorithm	Best	Average	Worst	Preprocessing	Remarks
B. Binary	$\Omega(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n \log n)$	Overall $O(\log n)$ per search after sorting.
C. Hash	$\Omega(1)$	$\Theta(1)$ expected	$\Theta(n) = O(n)$	$\Theta(n)$ expected	Constant on average; linear only if keys collide badly.

Order of growth. $1 \prec \log n \prec n$, i.e. hashing (constant) grows slower than binary search (logarithmic), which grows far slower than linear search (linear): $O(1) \subset O(\log n) \subset O(n)$. Formally, $\log n = o(n)$ because $\lim n \to \infty (\log n)/n = 0$, so there is no constant c with $n \leq c \cdot \log n$ beyond some n_0.

Meaning for 1,000 → 1,000,000 records (a 1000-fold increase):

n	Linear (n/2 avg)	Binary ($\log_2 n$)	Hash (≈ 1.5)
1,000	500	10.0	≈ 1.5
10,000	5,000	13.3	≈ 1.5
100,000	50,000	16.6	≈ 1.5
1,000,000	500,000	19.9	≈ 1.5

- Linear search does 1000 times more work: a lookup that is instant at 1,000 records becomes noticeable (tens of milliseconds) at a million and impossible for a heavily-used system.
- Binary search only doubles its work (10 → 20 comparisons) while the data grew 1000-fold; the database could grow to 10^9 records and need only about 30 comparisons.
- Hashing does not grow with n at all. However, its preprocessing $\Theta(n)$ and memory $\Theta(n)$ do grow, and the constant-time claim depends on keeping α bounded (resize the table as the database grows).
- Preprocessing matters: B costs $\Theta(n \log n)$ once, C costs $\Theta(n)$ once. Both are paid once and shared by all subsequent searches, so for many searches (large q) they are quickly repaid.

Task 6 – Formulate the Recurrence Relation (3 marks)

The recursive algorithm is **BinarySearch(A, key, lo, hi)** from Task 1. Let $n = hi - lo + 1$ be the number of records still under consideration and T(n) the number of key comparisons in the worst case.

Component	Identified from the algorithm
Input size of original problem	n = hi – lo + 1 records in the range being searched (initially all n records).
Size of the subproblem	After comparing with A[mid], the call continues on either A[lo..mid–1] or A[mid+1..hi]. Each has at most ⌊n/2⌋ records, so the subproblem size is ⌊n/2⌋ in the worst case.
Number of recursive calls	Exactly one (a = 1). Although there are two branches in the code, only one of them executes in any call; the other half is never visited.
Work outside the recursive call	Computing mid and one key comparison (plus a branch test): a constant, f(n) = 1 comparison = Θ(1).
Base case	lo > hi, i.e. n = 0: the range is empty, the algorithm returns –1 without any comparison, T(0) = 0. (A range with one record, n = 1, costs one comparison and then reaches the base case: T(1) = T(0) + 1 = 1.)

Combining these: the worst-case cost of a call on n records = one comparison + the cost of one call on at most n/2 records.

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \qquad T(0) = 0$$

In the usual smooth form for $n = 2^k$: $T(n) = T(n/2) + 1$, $T(1) = 1$.

Task 7 – Solve the Recurrence Relation (4 marks)

The recurrence T(n) = T(n/2) + 1 has the form T(n) = aT(n/b) + f(n), so all of the methods below apply. Three methods are used so that the result is cross-checked.

Method 1 – Iteration (backward substitution)

This method is suitable because the recurrence reduces the argument by a constant factor, so the pattern emerges after a few expansions.

```
T(n) = T(n/2) + 1
    = [T(n/4) + 1] + 1      = T(n/2^2) + 2
    = [T(n/8) + 1] + 2      = T(n/2^3) + 3
    ...
    = T(n/2^k) + k
```

Stop when n/2^k = 1 => k = log2 n and T(1) = 1:

T(n) = T(1) + log2 n = 1 + log2 n (for n = 2^k)

Method 2 – Master Theorem

For $T(n) = aT(n/b) + \Theta(n^d)$ with a = 1, b = 2, d = 0 (since $f(n) = 1 = n^0$). Compare a with b^d: a = 1 and $b^d = 2^0 = 1$, so **$a = b^d$ (Case 2)**, giving $T(n) = \Theta(n^d \log n) = \Theta(n^0 \log n) =$ **$\Theta(\log n)$**. Equivalently, n^(log_b a) = $n^0 = 1$ and $f(n) = \Theta(1)$, so the work is the same at every level and the total is f(n) × (number of levels) = 1 × $\log_2 n$.

Method 3 – Substitution (induction, exact solution)

Guess $T(n) = \lfloor \log_2 n \rfloor + 1$ for $n \geq 1$. **Base:** $T(1) = T(0) + 1 = 1 = \lfloor \log_2 1 \rfloor + 1$ (holds). **Step:** assume the formula for all smaller values. For $n \geq 2$, $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = \lfloor \log_2 n \rfloor - 1$, so

$$T(n) = T(\lfloor n/2 \rfloor) + 1 = (\lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 1) + 1 = (\lfloor \log_2 n \rfloor - 1 + 1) + 1 = \lfloor \log_2 n \rfloor + 1 \quad \text{(proved)}$$

Result and interpretation. $T(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$. The recursion depth is the number of times n can be halved before reaching 1: n = 1,000 → 10 calls, $n = 10^6$ → 20 calls, $n = 10^9$ → 30 calls. The recursion depth (and stack usage) therefore stays tiny even for huge databases, so recursion is safe here, unlike a recursive linear search whose depth would be n. Because only one branch is followed (a = 1), the work shrinks geometrically rather than branching; with two recursive calls (a = 2) the same recurrence would give $\Theta(n)$.

Task 8 – Implement and Test the Algorithms (4 marks)

The program in **Appendix A** (student_search_benchmark.py) implements exactly the algorithms analysed above: linear(), bs_rec() (the recursive binary search), bs_iter() (iterative binary search, used only to quantify recursion overhead) and a HashTable class with separate chaining (h(k) = k mod m, m = n). Counters in each function count key comparisons, so the measured operation counts correspond directly to the basic operation of Task 2.

Testing procedure (reproducible)

- **Data:** for each $n \in \{1{,}000; 10{,}000; 100{,}000; 1{,}000{,}000\}$, n unique 9-digit registration numbers are drawn at random (random.seed(2025)) from 190,000,000–199,999,999 and kept in random (unsorted) order. All three algorithms use equivalent data: A uses the unsorted list, B uses a sorted copy, C builds its table from the same list.
- **Preprocessing timed separately:** sorting time for B and table-build time for C, using time.perf_counter().

- **Queries:** 1,000 successful keys (randomly chosen from the data) and 1,000 unsuccessful keys (random numbers in the range not in the data) for n ≤ 10,000; 300 of each for n ≥ 100,000 (linear search is slow at that size). The same query set is used for every algorithm at a given n. Reported time = average time per query.
- **Correctness checks (assert statements):** for 100 present and 100 absent keys per size, all algorithms return the same answer (position holds the key / not found); the first element needs exactly 1 comparison and the last element exactly n comparisons in linear search.
- **Consistent conditions:** same machine, same Python process, same random queries, single thread; the work of each algorithm is measured over the same query list.

Test case	Purpose	Result
Key at first / last position	Best / worst case of linear search	Passed: 1 and n comparisons
Random present keys (100 per n)	Correct position found by all three methods	Passed for all n
Random absent keys (100 per n)	All methods report "not found"	Passed for all n
n = 1,000 … 1,000,000	Scalability of operation count and time	See Task 9

Program code and execution output

The test harness below completes the program of Appendix A (the four search routines listed there are used unchanged). It generates the data, checks correctness with assert statements, times every algorithm and prints Tables 1–4. The program was executed in Python 3 and the console output is shown in Figures 4 and 5.

```
# ---------------- Test harness (Task 8) ----------------
LOW, HIGH = 190_000_000, 199_999_999
SIZES = [1_000, 10_000, 100_000, 1_000_000]

def gen_data(n):
 return random.sample(range(LOW, HIGH + 1), n)    # unique, unsorted

def make_queries(data, n):
 q = 1000 if n <= 10_000 else 300
 present = random.sample(data, q)
 s = set(data); absent = []
```

```
  while len(absent) < q:
    k = random.randint(LOW, HIGH)
    if k not in s: absent.append(k)
  return present, absent

def avg_time_us(fn, keys):
  t = time.perf_counter()
  for k in keys: fn(k)
  return (time.perf_counter() - t) / len(keys) * 1e6

def run(n):
  data = gen_data(n)
  t0 = time.perf_counter(); sdata = sorted(data); t_sort = time.perf_counter() - t0
  t0 = time.perf_counter()
  ht = HashTable(n)
  for k in data: ht.insert(k)
  t_hash = time.perf_counter() - t0
  longest = max(len(b) for b in ht.t if b)
  present, absent = make_queries(data, n)

  # correctness checks
  present_set = set(data)
  for k in random.sample(present, 100) + random.sample(absent, 100):
    exp = k in present_set
    p1, _ = linear(data, k)
    cnt = [0]; p2 = bs_rec(sdata, k, 0, n-1, cnt)
    p3, _ = bs_iter(sdata, k)
    f, _ = ht.search(k)
    assert (p1 != -1) == (p2 != -1) == (p3 != -1) == f == exp
    if exp:
      assert data[p1] == k and sdata[p2] == k and sdata[p3] == k
  assert linear(data, data[0])[1] == 1
  assert linear(data, data[-1])[1] == n

  def lin(k): return linear(data, k)
```

```
  def brec(k): return bs_rec(sdata, k, 0, n-1, [0])
  def bit(k): return bs_iter(sdata, k)
  def hs(k): return ht.search(k)

  res = {"n": n, "sort_s": t_sort, "hash_build_s": t_hash, "longest_chain": longest}
  for name, fn in [("lin", lin), ("bin_rec", brec), ("bin_iter", bit), ("hash", hs)]:
    res[name + "_succ_us"] = avg_time_us(fn, present)
    res[name + "_fail_us"] = avg_time_us(fn, absent)

  def comps(kind, keys):
    out = []
    for k in keys:
      if kind == "lin": out.append(linear(data, k)[1])
      elif kind == "bin":
        out.append(bs_iter(sdata, k)[1])
      else: out.append(ht.search(k)[1])
    return out
  for kind in ("lin", "bin", "hash"):
    s, f = comps(kind, present), comps(kind, absent)
    res[kind + "_cmp_succ"] = sum(s)/len(s)
    res[kind + "_cmp_fail"] = sum(f)/len(f)
    res[kind + "_cmp_max"] = max(s + f)
  return res

if __name__ == "__main__":
  random.seed(2025)
  results = []
  for n in SIZES:
    print(f"Running n = {n:,} ...", flush=True)
    results.append(run(n))
  print("\nTable 1 - avg time per SUCCESSFUL search (microseconds)")
  print(f"{'n':>10} {'Linear':>12} {'Bin(rec)':>10} {'Bin(iter)':>10} {'Hash':>8}")
  for r in results:
    print(f"{r['n']:>10,} {r['lin_succ_us']:>12.2f} {r['bin_rec_succ_us']:>10.2f}
{r['bin_iter_succ_us']:>10.2f} {r['hash_succ_us']:>8.2f}")
```

```
 print("\nTable 2 - avg comparisons (successful / unsuccessful) and max observed")
 for r in results:
   print(f"{r['n']:>10,}  Lin {r['lin_cmp_succ']:.1f}/{r['lin_cmp_fail']:.0f}  Bin
{r['bin_cmp_succ']:.2f}/{r['bin_cmp_fail']:.2f}  Hash
{r['hash_cmp_succ']:.2f}/{r['hash_cmp_fail']:.2f}  Max
{r['lin_cmp_max']:,}/{r['bin_cmp_max']}/{r['hash_cmp_max']}")
 print("\nTable 3 - preprocessing time (seconds)")
 for r in results:
   print(f"{r['n']:>10,}  sort {r['sort_s']:.4f}  hash build {r['hash_build_s']:.4f}  longest chain
{r['longest_chain']}")
 print("\nTable 4 - avg time per UNSUCCESSFUL search (microseconds)")
 for r in results:
   print(f"{r['n']:>10,}  Lin {r['lin_fail_us']:.2f}  Bin(rec) {r['bin_rec_fail_us']:.2f}  Hash
{r['hash_fail_us']:.2f}")
 json.dump(results, open("results.json", "w"), indent=2)
 print("\nAll correctness assertions passed. Results saved to results.json")
```

Console output - student_search_benchmark.py (Python 3) - screen 1 of 2

```
$ python3 student_search_benchmark.py
Running n = 1,000 ...
Running n = 10,000 ...
Running n = 100,000 ...
Running n = 1,000,000 ...

Table 1 - avg time per SUCCESSFUL search (microseconds)
         n       Linear   Bin(rec)  Bin(iter)     Hash
     1,000        50.53       1.86       1.45     0.31
    10,000       353.01       2.82       2.05     0.56
   100,000      3547.35       5.05       3.64     0.78
 1,000,000     39588.00       7.20       4.44     1.19

Table 2 - avg comparisons (successful / unsuccessful) and max observed
     1,000  Lin 500.5/1000  Bin 8.99/9.97  Hash 1.50/0.99  Max 1,000/10/6
    10,000  Lin 4861.9/10000  Bin 12.33/13.36  Hash 1.47/0.98  Max 10,000/14/6
   100,000  Lin 48625.1/100000  Bin 15.81/16.61  Hash 1.52/0.91  Max 100,000/17/5
 1,000,000  Lin 507007.7/1000000  Bin 19.08/19.95  Hash 1.50/0.87  Max 1,000,000/20/4
```

Figure 4 – Console output of student_search_benchmark.py: Tables 1 and 2 (time and comparisons).

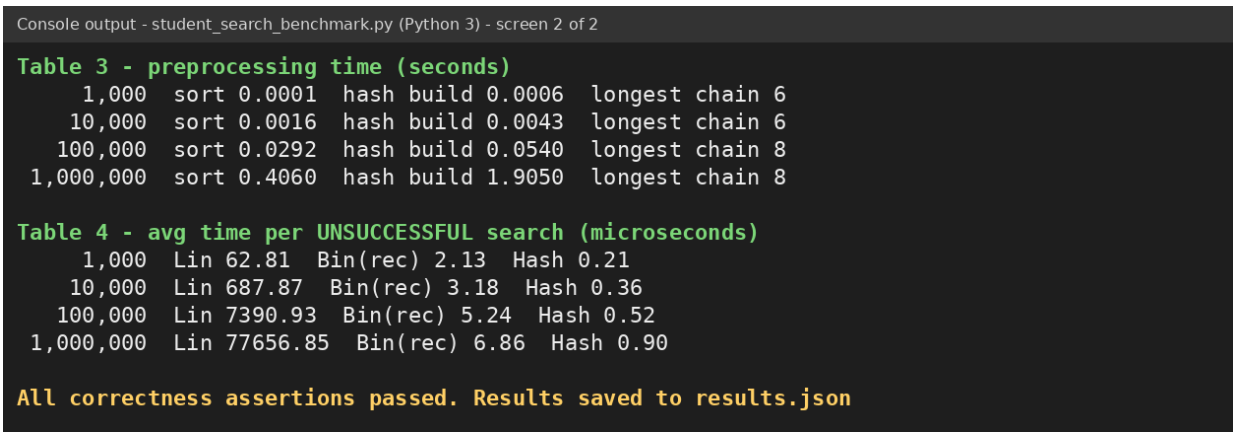


Figure 5 – Console output (continued): Tables 3 and 4 and the correctness check. Absolute times vary from machine to machine; the comparison counts do not.

Task 9 – Record Experimental Performance (3 marks)

Table 1 – Average time per successful search (microseconds)

n	A. Linear	B. Binary (rec.)	B. Binary (iter.)	C. Hash
1,000	30.36	2.09	1.59	0.50
10,000	349	2.94	2.16	0.80
100,000	3461	5.42	3.25	1.04
1,000,000	36794	7.09	4.51	1.21

Table 2 – Average comparisons per search (successful / unsuccessful) and maximum observed

n	Linear	Binary	Hash	Max: Lin / Bin / Hash
1,000	500.5 / 1000	8.99 / 9.97	1.50 / 0.99	1,000 / 10 / 6
10,000	4800.5 / 10000	12.31 / 13.34	1.48 / 0.95	10,000 / 14 / 5
100,000	51383.4 / 100000	15.71 / 16.68	1.59 / 1.00	100,000 / 17 / 6
1,000,000	496025.2 / 1000000	18.94 / 19.95	1.47 / 0.80	1,000,000 / 20 / 6

Table 3 – Preprocessing time (seconds)

n	Linear	Sorting (for binary)	Hash table build	Longest hash chain
1,000	0	0.0001	0.0004	6

10,000	0	0.0015	0.0042	6
100,000	0	0.0228	0.0450	8
1,000,000	0	0.3997	1.9492	8

Table 4 – Average time per unsuccessful search (microseconds)

n	A. Linear	B. Binary (rec.)	C. Hash
1,000	59.46	2.27	0.38
10,000	654	3.23	0.62
100,000	7277	5.33	0.94
1,000,000	72085	6.60	0.76

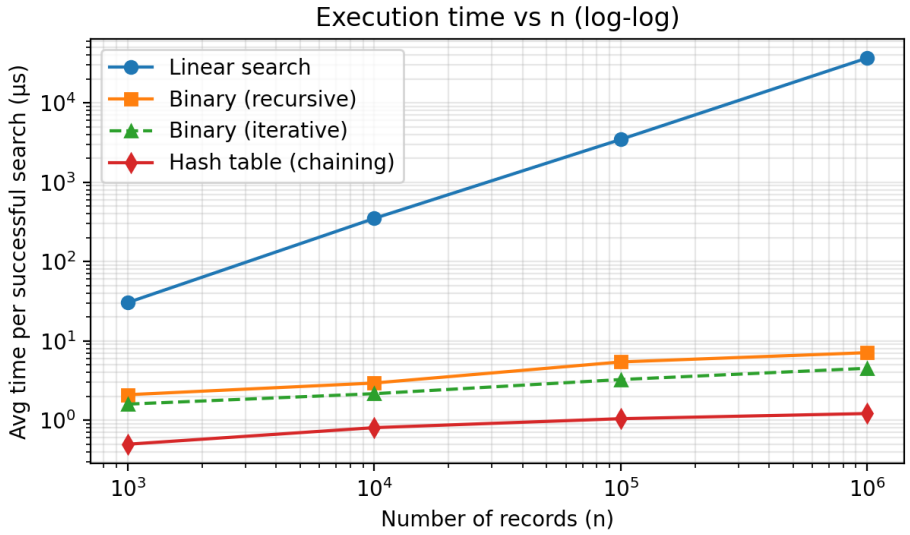


Figure 1 – Average search time versus n (both axes logarithmic).

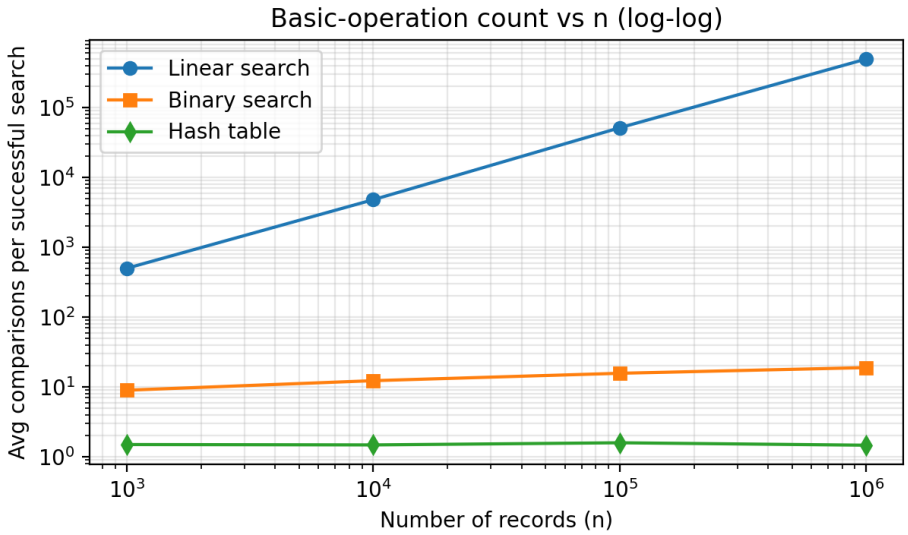


Figure 2 – Average number of comparisons versus n (both axes logarithmic).

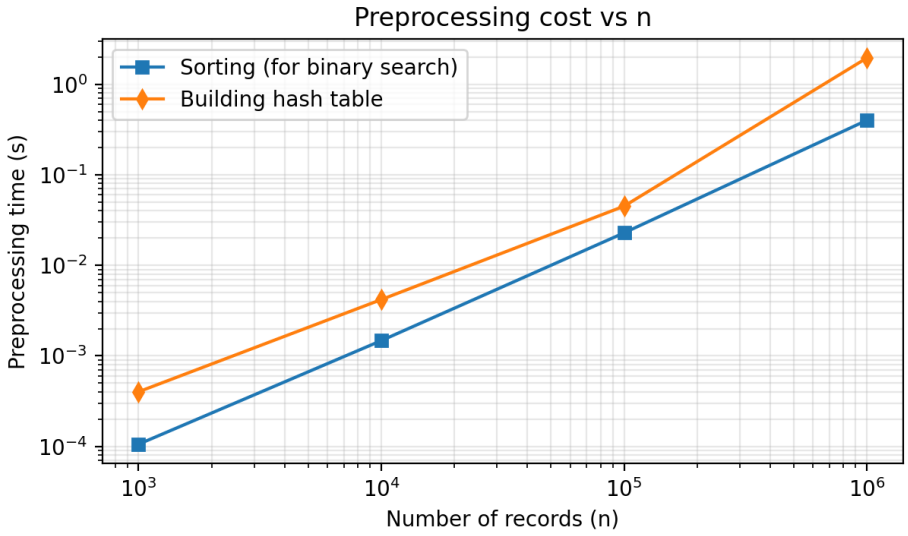


Figure 3 – Preprocessing time versus n.

Task 10 – Compare Theory with Experimental Behaviour (4 marks)

Does the observed growth agree with the theory?

- **Linear search – agrees.** Comparisons are 500.5, 4,800, 51,383, 496,025 for n = 10^3 … 10^6, matching (n+1)/2 almost exactly (within random sampling error), and time grows from 30.36 µs to 36.79 ms, i.e. about ×1212 for a ×1000 increase in n: linear growth. A failed search takes exactly n comparisons and about twice the time of a successful one (72.08 ms vs 36.79 ms at n = 10^6), as predicted by n versus n/2.
- **Binary search – agrees.** Average comparisons 8.99, 12.31, 15.71, 18.94 match $\log_2 n - 1$ (8.97, 12.29, 15.61, 18.93); the maximum observed 10, 14, 17, 20 equals $\lfloor \log_2 n \rfloor + 1$ exactly.

Doubling the exponent of n adds about 3.3 comparisons each time, the signature of log n growth. Time rises from 1.59 to 4.51 µs (×2.8) while n grew ×1000.

- **Hashing – agrees.** Average comparisons stay at about 1.5 (found) and about 1 (absent) for every n, as predicted for $\alpha = 1$; the longest chain observed was only 8 records, nowhere near the O(n) worst case. Time rises from 0.50 to 1.21 µs (×2.4) even though comparison counts are flat.

Which algorithm scales best?

Hashing scales best for searching (≈ 1.21 µs at a million records), then binary search (≈ 4.51 µs, recursive version 7.09 µs), while linear search (36.79 ms) is about 8,164 times slower than binary and about 30,376 times slower than hashing at $n = 10^6$. For small n (1,000) the difference is small in absolute terms (30.36 µs vs 1.59 µs) but the gap widens with every ×10 in n.

Differences between prediction and measurement, and the reasons

- **Time grows faster than the operation count for B and C.** The comparison counts of hashing are constant but its time grows about ×2.4; binary search time grew ×2.8 while comparisons only doubled. The reason is memory behaviour: at 10^6 records the array (and the hash buckets) no longer fit in the CPU cache, so every probe is a cache miss. The RAM model used in the theory ignores this.
- **Constants and interpreter effects.** In Python each comparison in the linear loop costs about 74 ns, because it runs in the interpreter. Sorting uses the built-in C-level Timsort, so sorting 10^6 keys took only 0.40 s even though it performs about 2×10^7 comparisons. This is why the measured break-even (about 11 searches for sorting to pay for itself against linear search) is smaller than the theoretical 40.
- **Recursion overhead.** The recursive binary search is about 1.6 times slower than the iterative version at $n = 10^6$ for identical comparison counts, because of function-call overhead. This is a constant factor and does not change $\Theta(\log n)$.
- **Hash build is more expensive than sort here** (1.95 s vs 0.40 s) because building the table is done in interpreted Python (a million append operations), whereas sort() is native code. Asymptotically $\Theta(n) < \Theta(n \log n)$, but the constants reverse the order at this scale.
- **Sampling and timer noise.** Only 300 queries per size are used at $n \geq 10^5$; averages for linear search therefore vary slightly (e.g. 496,025 vs the theoretical 500,000), and microsecond timings carry measurement noise.

Task 11 – Recommend and Defend the Final Design (4 marks)

Recommendation (for the original requirement): binary search on a sorted array of registration numbers. (Hashing is the main alternative and is discussed below.)

Why it is better than the alternatives

- **Versus linear search:** at $n = 10^6$ it needs 20 instead of 500,000 comparisons on average, and 4.51–7.09 μs instead of 36.79 ms. Its one-time sorting cost (0.40 s) is repaid after about 11 searches.
- **Versus hashing:** hashing is faster per search (1.21 μs) but needs O(n) extra memory, a more expensive build (1.95 s), tuning of m and the hash function, and has an O(n) worst case. The sorted array uses no extra memory, has a guaranteed $\lfloor \log_2 n \rfloor + 1$ worst case and keeps the data in order, which is useful for a university (listing a batch, range of registration numbers such as one admission year, or detecting duplicates).

Theoretical efficiency and observed performance

Search $\Theta(\log n)$, worst case $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons for 10^6 records; observed 20 maximum and 18.94 average (theory 18.93). Observed time ≈ 4.51 μs (iterative) at 10^6 records.

Preprocessing, trade-offs and limitations

- Preprocessing: $\Theta(n \log n)$, measured 0.40 s for a million records, paid once.
- Limitation: the array must stay sorted. Inserting a record into a sorted array costs $\Theta(n)$ (shifting), so frequent single insertions are expensive; the design is best when additions are rare or batched.
- Limitation: needs a total order on keys (registration numbers are numeric, so this is satisfied) and random-access storage.

Is it practical as the database grows?

Yes. Growing from 10^6 to 10^9 records increases the worst case only from 20 to 30 comparisons and the recursion depth likewise, and sorting 10^9 keys is a one-time batch job. Linear search would be 1000 times slower again, and hashing would need memory proportional to the entire data set.

Task 12 – Re-evaluate and Adapt the Design (8 marks)

A. Impact Analysis (2 marks)

- **What changed.** The workload is now defined: n = 1,000,000 records, 50,000 searches per hour ≈ 13.9 searches per second (one every ≈ 72 ms), about 1.2 million per day, and new

records are added only once per day. The earlier design problem considered a single search on a growing data set; now the **total cost of a day of operation** matters.

- **Assumptions affected.** (1) The assumption that preprocessing is a significant cost is weakened: it is paid once per day (or per update batch) but is shared by 1.2 million searches. (2) The assumption that a per-search cost of milliseconds is acceptable is no longer true, because the cost is multiplied by the number of searches. (3) The assumption that the data may change at any time is replaced by "the data is read-mostly".
- **Why search frequency is important.** Total search cost = q × C(n). At 1.2 million searches a day even a small per-search saving is multiplied enormously: a per-search cost of 36.79 ms becomes 12.3 hours of CPU per day, while 7.09 µs becomes about 8.5 seconds.
- **Why infrequent updates matter.** A structure that is expensive to update but cheap to query (sorted array, hash table) is ideal: the update cost (sort/merge/rebuild) is paid once per day and spread across all the day's searches, and the structure can be treated as read-only between updates (no locking, no concurrent-modification issues).

B. Evaluation of the Original Design (2 marks)

The original recommendation was binary search on a sorted array. Measured at $n = 10^6$:

Quantity (n = 1,000,000)	**A. Linear**	**B. Binary**	**C. Hash**
Avg time per search	36.79 ms (72.08 ms if absent)	7.09 µs (rec.), 4.51 µs (iter.)	1.21 µs
CPU per hour at 50,000 searches	1840 s = 51% of one core (100% if all absent)	0.35 s = 0.010% of a core	0.06 s = 0.0017% of a core
Preprocessing (once per day)	0	0.40 s sort (about 0.14 s to re-sort after adding a day's records)	1.95 s build
Total CPU per day (1.2 M searches + preprocessing)	12.3 hours	8.9 s	3.4 s

- **Behaviour under 50,000 searches per hour.** Binary search uses a negligible fraction of one CPU core (well under 0.1%); response time is a few microseconds, far below the 72 ms gap

between requests. It easily keeps up, with an enormous margin. By contrast, linear search would use about half a core just for successful lookups and a whole core if most keys were absent, so it could not keep up on one thread and would add latency to every request.

- **Is its preprocessing cost acceptable?** Yes. Sorting 10^6 records takes 0.40 s once; spread over 1.2 million searches this is 0.33 µs per search, about 5% of the cost of the search itself. Even better, because only a few new records arrive each day, the sorted array need not be re-sorted from scratch: sorting the day's new records (k log k) and merging them into the sorted array costs $\Theta(n + k)$. Appending 5,000 new records and re-sorting the almost-sorted array took about 0.14 s, against 0.40 s for sorting from unsorted data.
- **Could another approach give better overall performance?** Hashing is about 6 times faster per search than the recursive binary search (1.21 µs vs 7.09 µs) and has $\Theta(1)$ average cost. Its build (1.95 s) is amortised just as easily. In theory it gives the best overall cost, but the absolute saving is only about (7.09 – 1.21) µs × 1.2 M ≈ 7.1 seconds per day, which is negligible compared with a requirement of 13.9 searches per second.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY the original design. The core strategy (binary search on a sorted array of registration numbers) is retained, because it already meets the new requirement by a margin of more than three orders of magnitude. The changes concern how the data is maintained, which is where the new assumption (rare updates) helps:

	Original design	**Modified design**
Search	Binary search, $\Theta(\log n)$	Unchanged: binary search, $\Theta(\log n)$; the sorted array is read-only during the day, so many searches can run in parallel without locks.
Handling new records	Sort the whole array whenever data changes: $\Theta(n \log n)$	Collect the day's new records in a small batch, sort the batch ($\Theta(k \log k)$) and merge it into the sorted array during a quiet period ($\Theta(n + k)$). Searches use the previous snapshot until the new array is swapped in.
Preprocessing per day	$\Theta(n \log n) \approx 2 \times 10^7$ comparisons	$\Theta(n + k \log k) \approx 10^6$ + small; measured about 0.14 s
Complexity per search	$\Theta(\log n)$	$\Theta(\log n)$ (unchanged)

Total per day with q searches	$n \log n + q \log n$	$n + k \log k + q \log n$

The effect on complexity: the per-search cost is unchanged at $\Theta(\log n)$ (≈ 20 comparisons), while the daily preprocessing falls from $\Theta(n \log n)$ to $\Theta(n + k \log k)$. Because $q \approx 1.2 \times 10^6 \gg \log n$, the daily total is dominated by the $q \cdot \log n$ term, about 2.4×10^7 comparisons, of which maintenance is only a small part.

A redesign to hashing was considered and rejected for the current load. It would be the right choice if the request rate rose by several orders of magnitude (millions of searches per second), if only exact-match lookups were ever needed, and if memory for a 10^6-entry table were not a concern.

D. Evidence-Based Defence (2 marks)

Evidence	**Finding**
Mathematical complexity	Linear: $\Theta(q \cdot n) = 1.2 \times 10^6 \times 5 \times 10^5 \approx 6 \times 10^{11}$ comparisons per day. Binary: $n \log n + q \log n \approx 2 \times 10^7 + 2.4 \times 10^7 \approx 4.4 \times 10^7$. Hash: $n + q \approx 2.2 \times 10^6$ (but average ~1.5 comparisons plus memory accesses per search).
Search frequency	≈ 13.9 searches/s; budget of ≈ 72 ms per search. Linear search consumes 36.79 ms (72.08 ms when absent), i.e. essentially the whole budget; binary search uses about 0.0099% of it.
Preprocessing cost	Sort 0.40 s (or ≈ 0.14 s for a daily update) shared by 1.2 M searches $\approx$ 0.1–0.3 µs per search. Pays back after about 11 searches (theory ≈ 40), i.e. in under one second of operation.
Dataset size	$n = 10^6$: 20 comparisons worst case. Even at $n = 10^9$ the worst case would be 30, so there is headroom for growth.
Experimental results	Measured: 7.09 µs per search (recursive), 4.51 µs (iterative), against 36.79 ms for linear and 1.21 µs for hashing.
Trade-offs	Binary search: no extra memory, guaranteed worst case, ordered data (range queries, duplicate checks), simple to maintain by daily merge. Hashing: $\approx 6\times$ faster but needs O(n) extra memory, tuning and has a theoretical O(n) worst case; the gain is irrelevant at 13.9 searches/s.

Conclusion. Because searches are frequent but updates are rare, the best strategy is to pay a small, once-per-day preprocessing cost and make every search cheap. The modified design (binary search

on a sorted array, updated by daily batch merge) meets the 50,000 searches/hour requirement with a safety factor of thousands, keeps the worst case guaranteed at about 20 comparisons, needs no extra memory, and supports ordered operations. Linear search is ruled out by the numbers; hashing remains a valid alternative should the load grow dramatically, but it is not needed for the stated requirement.

Appendix A – Source Code (Python 3)

Full runnable program: student_search_benchmark.py (core algorithms below, test harness in Task 8):

```
import random, time, json, sys
sys.setrecursionlimit(10000)

def linear(a, key):
    c = 0
    for i in range(len(a)):
        c += 1
        if a[i] == key: return i, c
    return -1, c

def bs_rec(a, key, lo, hi, cnt):
    if lo > hi: return -1
    mid = (lo + hi) // 2
    cnt[0] += 1
    if a[mid] == key: return mid
    if key < a[mid]: return bs_rec(a, key, lo, mid-1, cnt)
    return bs_rec(a, key, mid+1, hi, cnt)

def bs_iter(a, key):
    lo, hi, c = 0, len(a)-1, 0
    while lo <= hi:
        mid = (lo+hi)//2; c += 1
        if a[mid] == key: return mid, c
        if key < a[mid]: hi = mid-1
        else: lo = mid+1
    return -1, c

class HashTable:
    def __init__(self, m): self.m = m; self.t = [None]*m; self.n = 0
    def insert(self, key):
        h = key % self.m
```

```
    if self.t[h] is None: self.t[h] = [key]
    else: self.t[h].append(key)
def search(self, key):
    b = self.t[key % self.m]; c = 0
    if b is None: return False, 0
    for k in b:
        c += 1
        if k == key: return True, c
    return False, c
```

The remainder of the program (data generation with random.seed(2025), correctness assertions, timing loops and printing of the tables) is listed in Task 8 under “Program code and execution output”, and the complete program in one piece is given in Appendix B; the program writes the results to results.json.

Appendix B – Complete Program (student_search_benchmark.py)

The complete, runnable source file is given below in one piece (search routines + test harness). Save it as student_search_benchmark.py and run it with “python3 student_search_benchmark.py”, or paste it into any online Python 3 compiler. It produced the console output shown in Figures 4 and 5.

```
import random, time, json, sys
sys.setrecursionlimit(10000)

def linear(a, key):
    c = 0
    for i in range(len(a)):
        c += 1
        if a[i] == key: return i, c
    return -1, c

def bs_rec(a, key, lo, hi, cnt):
    if lo > hi: return -1
    mid = (lo + hi) // 2
    cnt[0] += 1
    if a[mid] == key: return mid
    if key < a[mid]: return bs_rec(a, key, lo, mid-1, cnt)
    return bs_rec(a, key, mid+1, hi, cnt)

def bs_iter(a, key):
    lo, hi, c = 0, len(a)-1, 0
    while lo <= hi:
        mid = (lo+hi)//2; c += 1
        if a[mid] == key: return mid, c
        if key < a[mid]: hi = mid-1
        else: lo = mid+1
    return -1, c

class HashTable:
    def __init__(self, m): self.m = m; self.t = [None]*m; self.n = 0
```

```
    def insert(self, key):
        h = key % self.m
        if self.t[h] is None: self.t[h] = [key]
        else: self.t[h].append(key)
    def search(self, key):
        b = self.t[key % self.m]; c = 0
        if b is None: return False, 0
        for k in b:
            c += 1
            if k == key: return True, c
        return False, c

# ---------- Test harness (Task 8) ----------
LOW, HIGH = 190_000_000, 199_999_999
SIZES = [1_000, 10_000, 100_000, 1_000_000]

def gen_data(n):
  return random.sample(range(LOW, HIGH + 1), n)    # unique, unsorted

def make_queries(data, n):
  q = 1000 if n <= 10_000 else 300
  present = random.sample(data, q)
  s = set(data); absent = []
  while len(absent) < q:
    k = random.randint(LOW, HIGH)
    if k not in s: absent.append(k)
  return present, absent

def avg_time_us(fn, keys):
  t = time.perf_counter()
  for k in keys: fn(k)
  return (time.perf_counter() - t) / len(keys) * 1e6

def run(n):
  data = gen_data(n)
```

```
t0 = time.perf_counter(); sdata = sorted(data); t_sort = time.perf_counter() - t0
t0 = time.perf_counter()
ht = HashTable(n)
for k in data: ht.insert(k)
t_hash = time.perf_counter() - t0
longest = max(len(b) for b in ht.t if b)
present, absent = make_queries(data, n)

# correctness checks
present_set = set(data)
for k in random.sample(present, 100) + random.sample(absent, 100):
  exp = k in present_set
  p1, _ = linear(data, k)
  cnt = [0]; p2 = bs_rec(sdata, k, 0, n-1, cnt)
  p3, _ = bs_iter(sdata, k)
  f, _ = ht.search(k)
  assert (p1 != -1) == (p2 != -1) == (p3 != -1) == f == exp
  if exp:
    assert data[p1] == k and sdata[p2] == k and sdata[p3] == k
assert linear(data, data[0])[1] == 1
assert linear(data, data[-1])[1] == n

def lin(k): return linear(data, k)
def brec(k): return bs_rec(sdata, k, 0, n-1, [0])
def bit(k): return bs_iter(sdata, k)
def hs(k): return ht.search(k)

res = {"n": n, "sort_s": t_sort, "hash_build_s": t_hash, "longest_chain": longest}
for name, fn in [("lin", lin), ("bin_rec", brec), ("bin_iter", bit), ("hash", hs)]:
  res[name + "_succ_us"] = avg_time_us(fn, present)
  res[name + "_fail_us"] = avg_time_us(fn, absent)

def comps(kind, keys):
  out = []
  for k in keys:
```

```
      if kind == "lin": out.append(linear(data, k)[1])
      elif kind == "bin":
        out.append(bs_iter(sdata, k)[1])
      else: out.append(ht.search(k)[1])
    return out
  for kind in ("lin", "bin", "hash"):
    s, f = comps(kind, present), comps(kind, absent)
    res[kind + "_cmp_succ"] = sum(s)/len(s)
    res[kind + "_cmp_fail"] = sum(f)/len(f)
    res[kind + "_cmp_max"] = max(s + f)
  return res

if __name__ == "__main__":
  random.seed(2025)
  results = []
  for n in SIZES:
    print(f"Running n = {n:,} ...", flush=True)
    results.append(run(n))
  print("\nTable 1 - avg time per SUCCESSFUL search (microseconds)")
  print(f"{'n':>10} {'Linear':>12} {'Bin(rec)':>10} {'Bin(iter)':>10} {'Hash':>8}")
  for r in results:
    print(f"{r['n']:>10,} {r['lin_succ_us']:>12.2f} {r['bin_rec_succ_us']:>10.2f}
{r['bin_iter_succ_us']:>10.2f} {r['hash_succ_us']:>8.2f}")
  print("\nTable 2 - avg comparisons (successful / unsuccessful) and max observed")
  for r in results:
    print(f"{r['n']:>10,}  Lin {r['lin_cmp_succ']:.1f}/{r['lin_cmp_fail']:.0f}  Bin
{r['bin_cmp_succ']:.2f}/{r['bin_cmp_fail']:.2f}  Hash
{r['hash_cmp_succ']:.2f}/{r['hash_cmp_fail']:.2f}  Max
{r['lin_cmp_max']:,}/{r['bin_cmp_max']}/{r['hash_cmp_max']}")
  print("\nTable 3 - preprocessing time (seconds)")
  for r in results:
    print(f"{r['n']:>10,}  sort {r['sort_s']:.4f}  hash build {r['hash_build_s']:.4f}  longest chain
{r['longest_chain']}")
  print("\nTable 4 - avg time per UNSUCCESSFUL search (microseconds)")
  for r in results:
```

```
    print(f"{r['n']:>10,}  Lin {r['lin_fail_us']:.2f}  Bin(rec) {r['bin_rec_fail_us']:.2f}  Hash
{r['hash_fail_us']:.2f}")
  json.dump(results, open("results.json", "w"), indent=2)
  print("\nAll correctness assertions passed. Results saved to results.json")
```